Синхронизация доступа к разделяемому ресурсу

набор из 2-х семафоров 0 | 1

1. Серверный процесс создает разделяемый ресурс и набор из 2-х семафоров и устанавливает их в значения 0 | 1
2. Серверный процесс ожидает данных от клиента (выполняется попытка открытия семафора функцией `semop(-1, 0)`)

3. Клиенты хотят получить доступ к разделяемому ресурсу (`semop(0, -1)`): 0 | 0
4. Получивший доступ клиент пишет запрос в разделяемый ресурс`semop(1, 0)`: 1 | 0
5. Клиент, который разблокировал сервер, выполняет `semop(-1,-1)`.
6. Процесс-сервер получает доступ к разделяемому ресурсу и обрабатывает запрос клиента `semop(1,1)`: 1 | 1 Серверный процесс переходит в режим ожидания, выполняя `semop(-1, 0)`
7. Клиент, который ожидает ответ от серверного процесса разблокируется, прочитает ответ сервера. И клиент выполняет вызов `semop(0,1)`: 0 | 1

System V

- get - методы для получения доступа к соответствующему средству синхронизации (`shmget`, `msgget`, `semget`)
- ... ctl - методы уровня управления средствами синхронизации
- методы чтения/записи (модификация) средства синхронизации

Очередь событий SystemV

07/03/2026

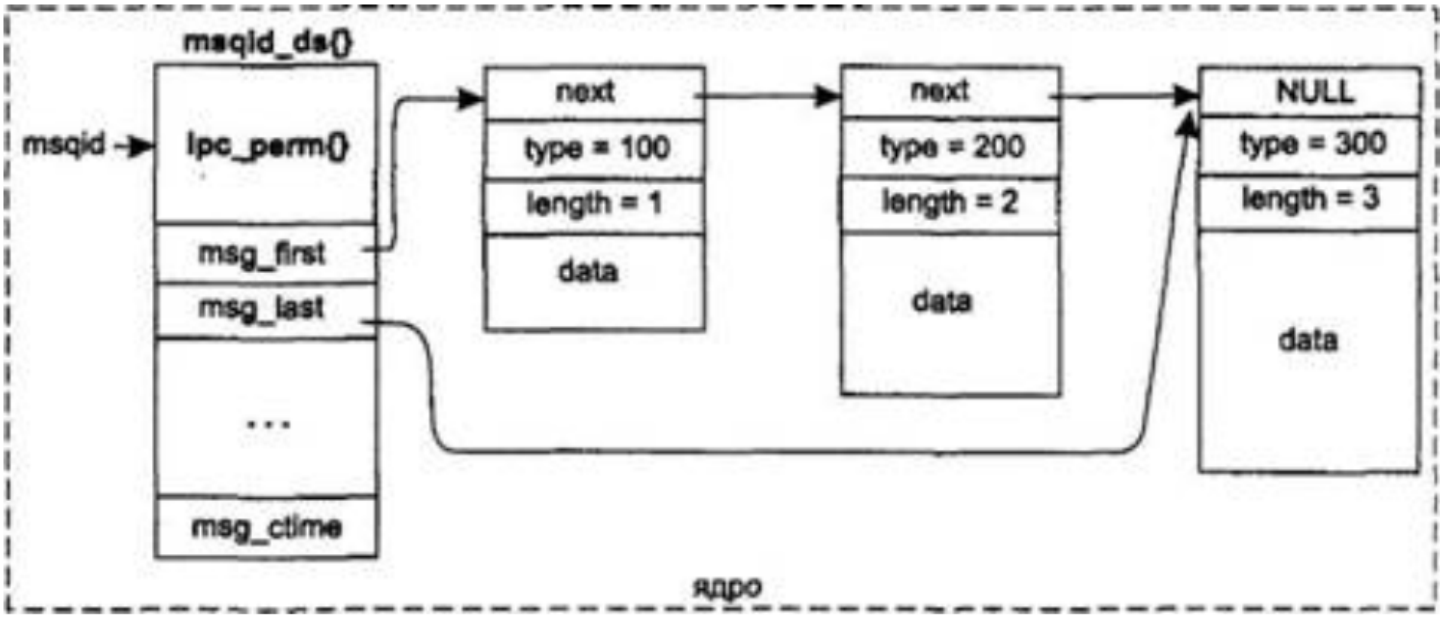

Figure 1: Структура очереди system V в ядре

Лекция

Атрибуты очереди сообщений

- Имя(целое число)
- UID, GID создание очереди
- UID, GID назначение владельца
- Права доступа к очереди сообщений
- время и индефикаатор процесса, который последний передал сообщение в очередь
- время и индефикаатор процесса, который последний прочитал сообщение
- указатель на сообщение

Сообщение

- Данные сообщения
- Тип сообщения

Создание/получение доступа(открытие) очереди сообщений

```
int msget(ley_t  key, int flog)
// key - интреприматор(имя) очереди сообщений
// flag - флаг
```

Флаги

- IPC_CREAT|IPC_EXCL - гаррантированное создание новой очереди сообщений IPC_CREAT|0777 - разрешить все

Отправка(запись) сообщения в очередь

```
int msgsnd(ind msgfd, cons void* msgPtr, int len, int flag)
// msgfd - целое число, которое плученно от msget
// msgPtr - указатель на переменную типа struct msbuf
// len - длина в байтах поля mtext, на который указывает msgPtr
struct msbuf
{
    long mtype; // тип сообщения
    char mtext[MSGMAx] // сообщение
}
```

IPC_NOWAIT - указание на то, что вызов не является блокирующим

Получение(чтение) сообщения из очереди

```
int msgrev(int msgfd, const void* mdgPrt, int len, int mtype, int flag)
// msgfd - целое число, которое полученно от msget
// mdgPrt - указатель на переменную, тип котрой похож на msgbuf
// len - размер, сообщение которое хотим считать
// mtype - тип, сообщения которое хотим считать
// flag - флаги
```

Смысл mtype

- 0 - считать из очереди самое мтарое значение любого типа
- положительное число - считать из очереди самое старое сообщение заданного типа
- отрицательное число - считать из очереди сообщение, тип которого меньше абсолютного значения mtype или равное ему. Если таких сообщений неколько, то считать самое старое

Управление очередью сообщений

```
int msgctl(int msgfd, int cmd, struct msgId_ds *mbufPtr)
// msgfd - целое число, которое полученно от msget
// cmd - команда
// msgId_ds * mbufPt - указатель на структуру с полями, которые описывают очередь
```

IPC_STAT - получение атрибутов очереди IPC_SET - установление атрибутов очереди IPC_RMID - удаление очереди

```
msgctl(msgfd, IPC_RMID, Null) // удаление очереди сообщений
```

Концепции работы с семафорами

```
int semget(key_t ley, int num_sem, int flag)
int semop(int semId, struct scmbuf *pBuf, int len)

struct sembuf{
    short sem_num;
    short sem_op;
    short sem_flag;
};
```

Флаг SEM_UNDO

```
int semdl(int semId, int num, int cmd, union semnum arg)

union semun{
    int val; // значение семафора
    struct semid_ds *buf; // управляющие парметры
```

```
    ushort *array; // массив значений семафора
}
```

- IPC_STAT - получение информации о работе семафоров
- IPC_SET - модификация управляющих парметров
- IPC_RMID - удаление наборов семафоров
- GETALL - получить текущее значение всех семафоров в наборе
- SETALL - установить все значения семафоров с помощью arg.array
- GETVAL - получить значение семафора с номером num (arg не использ)
- SETVAL - установить самафор со значением num в значение arg.val
- GETNCNT -
- GETZCNT -

Концепции работы с разделяемой областью памяти

```
int shmget(key_t key, int size, int flag)
void* shmat(int shmId, void*addr, int flag)
int shmdt(void* addr)
      shmctl -||-
```

- IPC_STAT
- IPC_SET
- IPC_RMID - удалить можно только неиспользумую разделяемой памяти

Интернет (очень похоже на лецию)

Подробный вариант лекции

Файлы. Файловая система (VFS - виртуальная файловая система).

(315 стр. учебника) 04/04/2025

- Файлы
- Описанией файлов (метаданные)
- иерархическая структура

```
    App              VFS              FS
┌──────────┐    ┌──────────┐    ┌──────────┐
│ Call func │ -> │          │ -> │          │
└──────────┘    └──────────┘    └──────────┘
```

Объекты VFS

- Суперблок
- Файловый индекс (информация о файле)
- Элемент каталога
- Открытый файл, который связан с некоторым процессом

В каждом объекте содержаться операции (действия), которые можно выполнить над данным объектом.

Виртуальные методы VFS

- super_operations
- inode_operations
- dentry_operations
- file_operations

Операции суперблока

```
struct super_operations {
    void (*destroy_inode) (struct inode *);
    int (* write_super) (struct super_block *);
    ...
};

struct super_block {
    ...
    struct super_operations* s_op;
```

```
    ...
};

struct super_block* sb;
sb->s_op->write_super(sb);
```

Работа с файлами

Атрибуты:

- тип файла
- права доступа
- количество жестких ссылок
- индификатор владельца
- индификатор группы владельца
- размер файла в байтах
- время последнего действия
- время последней модификации
- время последнего изменеия атрибутов
- номер индексного дискриптора (inode number)
- индификатор файловой системы

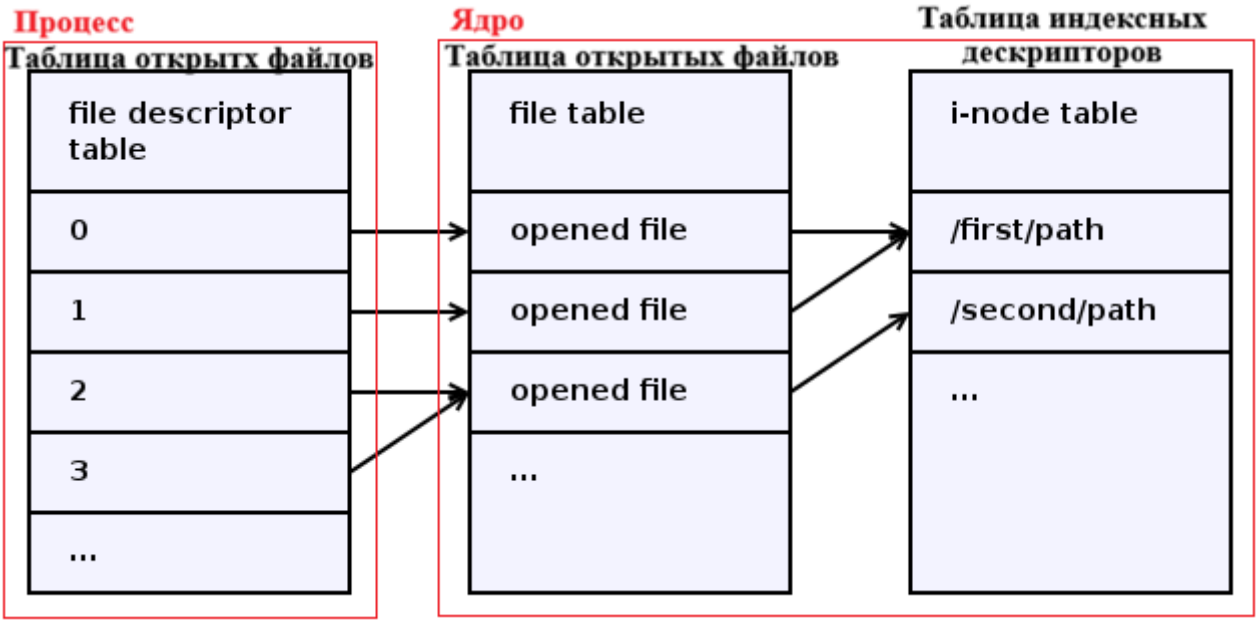

Figure 2: Схема

Последовательность шагов функции open

1. Ядро выполняет поиск в таблице дескрипторов файлов процесса первую незадействованную позицию. Номер (индекс) этой позиции является возвращаемым значением функции open.
2. Ядро выполняет поиск не занятой позиции в таблице открытых файлов ядра. УСЛ: Если свободная позиция найдена, то в найденную запись таблицы открытых файлов процесса заносится ссылка на найденную запись таблицы ядра.
    - В таблице открытых файлов в найденную позицию записывается ссылка на ту запись из таблицы inode, которая соответствует нужному файлу.
    - В таблице открытых файлов индексируется указатель текущей позиции чтения/записи.
    - В запись таблицы открытых файлов записывается режим открытия файла.
    - В записи таблицы открытых файлов изменяется счетчик ссылок на данный файл.
    - Запись счетчика ссылок индексного дескриптора увеличивается на 1.

Закрытие файла (вызов функции close())

1. Соответсвуящая запись в таблице открытых файлов процесса помечается как не используемая (незадействонная).
2. Ядром уменьшается значение счетсчика ссылок таблицы окрытых файлов на 1.
    - УСЛ: Если значение ссылок отлично от 0, то перейти к пункту 6.
3. Позиция таблицы открытых файлов помечается как незадейсвоная.

4. Значение счетчика ссылок в соответствующей записи таблицы индексных дескрипторов файла уменьшается на 1.
    - УСЛ: Если значение счетчика ссылок отлично от 0, то перейти к пункту 6.
5. Если значение счетчика ссылок в записи таблицы индексных дескрипторов отлично от 0, то происходит возвращение управления вывовшему процессу и с кодом (0).
    - Иначе ядро помечает соответсвкющую запись в таблице индексных дескрипторов, как неадействаную и физически удаляет память, котроя выделена на диске.
6. Ядро возвращает управление вызовавшему процессу и возвращает хкод успешного выполнения (0).

Подробный вариант лекции